

XAI 기반 퇴직연금 포트폴리오 진단 서비스

"M-ROBO는 최적화를 했다. 우리는 그 이유를 설명한다."
기존 로보어드바이저의 블랙박스를 XAI로 여는 핀테크 AX 프로젝트

프로젝트 기간: 4주 | 2025

목 차

01	문제 정의 — 왜 지금인가
02	시장 현황 및 경쟁 분석
03	프로젝트 차별화 포인트
04	데이터셋 확보 전략
05	분석 파이프라인
06	모델링 상세
07	최종 결과물 (Streamlit 프로토타입)
08	4주 실행 계획

01 문제 정의 — 왜 지금인가

1.1 핵심 문제 — 58조원이 방치되고 있다

83.3%

DC형 방치 비율

58조원

방치 적립금 규모

1.1억원

26년 방치 기회비용

30%p

수익률 상·하위 격차

DC형·IRP 가입자는 스스로 운용 지시를 내려야 하지만, 현실은 대부분 방치되고 있다. 무관심·무지·귀찮음으로 인해 적립금의 83.3%(58조원)가 연 1~2%대 원리금보장형에만 묶여 있으며, 인플레이션 감안 시 실질 수익률은 제로에 가깝다.

실제 피해 사례 — A씨(58세)는 26년간 원리금보장형(연 2.5%)에 방치. 적립금 약 1억 3,700만원. 주식 60%/채권 40%(연 6%)로 운용했다면 약 2억 4,800만원. 기회비용 1억 1,000만원 손실.

1.2 기존 플랫폼의 3대 구조적 한계

한계	현상	기존 대응의 공백
블랙박스 추천	AI가 포트폴리오 제시하나 '왜'에 대한 설명 없음. 신뢰 저하 및 이탈 유발	삼성·미래에셋 로보어드바이저 모두 결과만 제시. XAI 사례 없음
방치 비용 비가시화	방치 시 개인화된 기회비용을 수치로 보여주는 서비스 전무	TDF·디폴트옵션은 방치 해소 시도이나 기회비용 정량화 없음
매크로 미반영	금리 인상기에도 정적 MPT 모델 그대로 적용. 2022~2023 채권 -20% 대응 실패	대부분 ECOS 금리 연동 없음. 과거 수익률만 참고

1.3 Why Now

- 퇴직연금 적립금 2025년 400조원 돌파, 2035년 1,000조원 전망
- 2025년 4월 미래에셋 M-ROBO 출시 — AI 퇴직연금 운용 시장 공식 개막, 시장 검증 완료
- M-ROBO도 해결하지 못한 XAI(설명 가능성) 공백이 명확히 드러남
- 퇴직연금 의무화 법안 2025년 발의 — 향후 가입자 폭발적 증가 예상
- 금융당국 AI 가이드라인 7대 원칙에서 설명 가능성(XAI) 강조 — 규제 방향과 일치

02 시장 현황 및 경쟁 분석

2.1 최신 서비스 현황 (2025년 기준)

서비스	특징	XAI	방치 진단	매크로
미래에셋 M-ROBO (2025.04 출시)	AI 알고리즘 12개, 성향·연령 기반 자동 포트폴리오 + 동적 리밸런싱			부분적
미래에셋증권 RA (AUM 3.9조)	코스콤 테스트베드 통과. 알림 기반 리밸런싱 제안			
NH DNA 퇴직연금 (수익률 28.7%)	연환산 수익률 1위. 20~30대 가입비중 46.7%			
핀트·불리오·에임	간편 성향 진단 → 3유형 분류 → 포트폴리오 추천			
우리 프로젝트 ★	MPT 최적화 + SHAP 변수별 기여도 시각화 + 방치 기회비용 수치화			

포지셔닝 전략 — M-ROBO 등장은 '경쟁자 출현'이 아닌 '시장 검증'이다. M-ROBO가 포트폴리오 자동화를 해결했다면, 우리는 블랙박스를 열어 사용자 신뢰를 확보하는 다음 단계(XAI)를 제안한다.

03 프로젝트 차별화 포인트

3.1 핵심 차별화 2가지

차별화 1 XAI 기반 설명 가능한 포트폴리오	차별화 2 방치 포트폴리오 기회비용 수치화
MPT로 최적 ETF 비중을 계산한 뒤, SHAP 값으로 각 변수(금리, 섹터 수익률, 변동성 등)가 ETF 비중 결정에 얼마나 기여했는지 막대 차트로 시각화. 기준: '채권 ETF 30% 담으세요' 우리: '금리 인상 국면이 채권 비중에 -15%p 기여, 국내주식 변동성이 +8%p 기여'	사용자가 현재 포트폴리오(예: 원리금보장형 100%)를 입력하면, 효율적 프론티어 상의 위치를 시각화하고 '30년 운용 시 기회비용 X,XXX만원'을 자동 계산. 발표 데모에서 30초 안에 '방치하면 은퇴 시 1억원 손해'라는 숫자를 출력

3.2 발표 스토리라인 (심사위원 관점)

Problem	DC/IRP 58조원 방치. 수익률 상위 1%(33%) vs 하위(3%) 30%p 격차. 개인은 이 비효율을 인식조차 못함
Why Now	2025년 4월 미래에셋 M-ROBO 출시로 AI 퇴직연금 시장 개막. 단 XAI는 여전히 공백
Solution	MPT 최적화 + SHAP 설명 + 방치 기회비용 수치화 3-레이어 Streamlit 프로토타입
Validation	3년 백테스팅: 최적 포트폴리오 vs 원리금보장형 vs 코스피200 수익률·샤프지수 비교
Next Step	마이데이터 API 연동 시 실계좌 기반 실시간 진단 확장. M-ROBO 다음 버전 포지셔닝

04 데이터셋 확보 전략

4.1 핵심 전제 — 개인 포트폴리오 데이터는 존재하지 않는다

개인 DC/IRP 포트폴리오 구성 데이터는 금융 개인정보이므로 공개되지 않는다. 이를 역으로 활용 — '개인 포트폴리오 효율성을 진단해주는 도구가 없다'는 사실 자체가 우리 프로젝트의 존재 이유(Problem Statement)가 된다.

4.2 실제 사용 데이터셋 (전체 무료·즉시 확보 가능)

데이터	출처	획득 방법	용도
국내 ETF 50종 일봉 (5년)	pykrx / KRX	pip install pykrx 후 코드 3줄	포트폴리오 최적화 핵심 입력값
기준금리·국채수익률 월별	한국은행 ECOS	ECOS Open API 키 발급 (즉시)	금리 국면 분류 변수
원리금보장형 평균 금리	공공데이터포털 근로복지공단	data.go.kr CSV 다운로드	방치 시나리오 벤치마크
퇴직연금 수익률 통계	공공데이터포털	CSV 직접 다운로드	문제 규모 수치화 (발표용)
퇴직연금 가능 ETF 목록	KRX 정보데이터시스템	무료 CSV 다운로드	투자 가능 ETF 필터링

4.3 방치 시나리오 데이터 (직접 설계)

개인 포트폴리오 공개 데이터 부재로, 대표 시나리오 10개를 직접 설계한다. '실제 사용자 페르소나 기반 분석'으로 발표 설득력을 높인다.

시나리오	나이	은퇴 목표	현재 포트폴리오	계산 결과
방치형 직장인	35세	65세 (30년)	원리금보장형 100%	기회비용 1.1억원
보수형 투자자	45세	65세 (20년)	원리금보장 70% + 국내주식 30%	기회비용 4,200만원
적극형 투자자	30세	60세 (30년)	국내주식 ETF 70% + 채권 30%	샤프지수 개선 분석
은퇴 임박형	58세	65세 (7년)	주식 ETF 80% (고위험)	MDD 리스크 경고
균형형 직장인	40세	65세 (25년)	S&P500; 50% + 채권 50%	효율적 프론티어 근접

데이터 확보 실행 순서 — 1) pip install pykrx 후 ETF 데이터 수집 (즉시 시작 가능) → 2) ECOS API 키 발급 (당일) → 3) 공공데이터포털 퇴직연금 수익률 CSV 다운로드 → 4) 방치 시나리오 10개 설계 (1주차)

05 분석 파이프라인

5.1 전체 파이프라인 구조 (6단계)

1단계 데이터 수집	pykrx: ETF 50종 5년치 일봉 ECOS: 기준금리·국채수익률 공공데이터: 원리금보장형 금리
2단계 전처리 & EDA	수익률 계산: $\ln(P_t/P_{t-1})$ 결측치 처리 공분산 행렬 계산 금리 국면 분류 (인상/하락/중립)
3단계 포트폴리오 최적화	scipy.optimize로 MPT 구현 효율적 프론티어 도출 (5,000개 랜덤) 최소분산·최대샤프 계산
4단계 XAI (SHAP)	RandomForest 메타 모델 학습 SHAP 값 계산 Waterfall Plot: ETF 비중 결정 이유 Top 5
5단계 기회비용 계산	입력: 나이·현재 포트폴리오·은퇴 목표 복리 시뮬레이션 30년 후 자산 차이 프론티어 위치 마킹
6단계 백테스팅 & 검증	2020~2024년 실제 데이터 검증 수익률·MDD·샤프지수 비교 금리 국면별 성과 분석

5.2 핵심 코드 구조 (의사코드)

```
# 1. ETF 데이터 수집
from pykrx import stock
df = stock.get_market_ohlc('20200101', '20241231', etf_ticker)

# 2. 수익률 및 공분산 계산
returns = np.log(prices / prices.shift(1)).dropna()
cov_matrix = returns.cov() * 252 # 연간화

# 3. 포트폴리오 최적화 (MPT)
from scipy.optimize import minimize
result = minimize(neg_sharpe, x0=weights,
constraints=[{'type': 'eq', 'fun': lambda x: sum(x)-1}],
bounds=[(0, 0.7)]*n_etf) # 70% 제한

# 4. SHAP 설명
import shap
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X)
shap.waterfall_plot(explainer.expected_value, shap_values[0])

# 5. 기회비용 함수
def opportunity_cost(cur_r, opt_r, years, principal):
    return principal*((1+opt_r)**years - (1+cur_r)**years)
```

06 모델링 상세

6.1 모델 구성 및 차별화

모델	입력 변수	출력	차별화
마코위츠 MPT (Baseline)	ETF 수익률, 변동성, 상관관계	효율적 프론티어, 최적 비중	기존 서비스와 동일 수준 베이스라인
금리 국면 반영 Dynamic MPT	ETF 데이터 + ECOS 기준금리 국면 (인상/하락/중립)	국면별 최적 비중 자동 조정	금리 인상기 채권 비중 자동 축소. 핵심 차별화
SHAP 메타 모델 (XAI)	금리, 섹터 수익률, 변동성, PER/PBR	변수별 기여도 Waterfall Plot	블랙박스 해소. '왜 이 비중인가' 근거 제시
기회비용 시뮬레이터	나이, 은퇴 목표, 현재 포트폴리오	30년 복리 기대자산, 기회비용	방치 대비 최적화 시 추가 수익 수치화

6.2 평가 지표

지표	계산식	기준	의미
샤프지수	$(R_p - R_f) / \text{연환산 표준편차}$	> 1.0 목표	위험 대비 초과수익. 높을수록 효율적
MDD (최대 낙폭)	Peak에서 Trough까지 최대 하락률	< -15% 경고 기준	최악 시나리오에서 얼마나 손실 보는가
기회비용	$FV(\text{최적}) - FV(\text{현재 포트폴리오})$	양수일수록 개선	방치 대비 최적화로 얻는 추가 자산
효율적 프론티어 거리	현재 포트폴리오와 프론티어 간 거리	0에 가까울수록 효율적	포트폴리오 비효율성의 시각적 지표

07 최종 결과물 — Streamlit 프로토타입

7.1 프로토타입 4개 화면 구성

화면 1 포트폴리오 진단 입력	나이·은퇴 목표·현재 포트폴리오 구성 슬라이더 입력 UI ETF 비중 직접 조정 또는 원리금보장형 % 입력 '진단하기' 버튼 → 즉시 분석 시작
화면 2 효율적 프론티어 시각화	5,000개 랜덤 포트폴리오 + 효율적 프론티어 곡선 Plotly 인터랙티브 차트 사용자 현재 포트폴리오 빨간 점 표시 최적 포트폴리오(최대 샤프·최소분산) 별표 표시
화면 3 SHAP 설명 (XAI 핵심)	Waterfall Plot: 이 포트폴리오 비중의 상위 5가지 이유 자동 출력 '금리 인상 국면이 채권 비중에 -15%p 기여' Force Plot: 각 변수의 영향 방향(+/-) 시각화
화면 4 기회비용 계산 리포트	현재 vs 최적 포트폴리오 30년 복리 시뮬레이션 라인 차트 '방치 시 X억원 vs 최적화 시 Y억원' 대비 카드 백테스팅 수익률 비교 (2020~2024) 막대 차트

7.2 기술 스택

레이어	라이브러리	역할
데이터 수집	pykrx, ECOS API (requests)	ETF 일봉, 기준금리 시계열 수집
데이터 처리	pandas, numpy	수익률 계산, 공분산 행렬, 결측치 처리
포트폴리오 최적화	scipy.optimize, PyPortfolioOpt	MPT 효율적 프론티어, 샤프지수 최대화
XAI	scikit-learn (RandomForest), shap	메타 모델 학습, SHAP 값 계산 및 시각화
시각화	plotly, matplotlib, seaborn	효율적 프론티어, SHAP 플롯, 백테스팅 차트
프로토타입	Streamlit	4개 화면 인터랙티브 웹앱 (로컬 실행)
백테스팅	backtrader 또는 직접 구현	2020~2024년 전략 성과 검증

08 4주 실행 계획

8.1 주차별 작업 계획

주차	주요 작업	산출물
1주차 데이터 세팅	pykrx·ECOS 환경 구축 ETF 50종 데이터 수집 수익률·공분산 계산 금리 국면 분류 설계 방치 시나리오 10개 설계	전처리 완료 DataFrame 금리 국면 레이블 시나리오 정의서
2주차 모델링	MPT 최적화 구현 (scipy) 효율적 프론티어 도출 금리 국면 반영 Dynamic MPT 기회비용 계산 함수	최적화 코드 효율적 프론티어 데이터 기회비용 계산 결과
3주차 XAI + 프로토타입	RandomForest 메타 모델 학습 SHAP 값 계산 및 시각화 Streamlit 4개 화면 구현 백테스팅 수행	SHAP 플롯 Streamlit 앱 (로컬 동작) 백테스팅 결과 차트
4주차 발표 준비	발표 자료 제작 (PPT) 데모 시나리오 스크립트 결과 수치 정리 및 검증 라이브 데모 리허설	발표 PPT 데모 스크립트 최종 프로토타입

성공 기준 (Definition of Done) — Streamlit 앱에서 '원리금보장형 100%' 입력 시 효율적 프론티어 위치, SHAP 설명, 30년 기회비용 수치가 30초 내에 출력될 것. 백테스팅에서 최적 포트폴리오의 샤프지수가 원리금보장형 대비 최소 2배 이상일 것.